Unit 4- Java Server Pages

4.1 JSP Overview

Definition

- **Java Server Pages (JSP)** is a **server-side web technology** used to create **dynamic web applications**.
- It allows embedding **Java code inside HTML** to generate dynamic content.
- JSP pages are converted into **Servlets** by the server (like Apache Tomcat).
- JSP runs on a web server like Apache Tomcat and converts JSP into a **Servlet internally**.

Why JSP is Used?

- To create dynamic web pages (e.g., login systems, dashboards)
- To separate **presentation (HTML)** from **business logic (Java)**
- To simplify servlet development

Features of JSP

1. **Easy Development** – Write HTML with Java code
2. **Platform Independent** – Runs on any OS with JVM
3. **Reusable Components** – Supports JavaBeans
4. **Implicit Objects** – Predefined objects like request, response
5. **Faster Development** – No need to write full servlet code
6. **Supports MVC (Model–View–Controller)Architecture**

➢ JSP Life Cycle

1. Translation Phase (JSP → Servlet)

- JSP file (index.jsp) is converted into a **Servlet (.java file)**
- This is done by the server(Apache Tomcat) automatically.
- **Example:**

  index.jsp → index_jsp.java

2. Compilation Phase

- The generated servlet file is compiled into a **class file (.class)**
- **Example:**

  index_jsp.java → index_jsp.class

3. Class Loading

- The compiled class is loaded into server memory
- Ready to execute

4. Initialization Phase

- Method: init() method called.
- Called **only once** when JSP is loaded.
- Used for: Initialization tasks

  Database connection setup

5. Execution Phase

- Method: _jspService()
- Called **every time user sends request**
- This method:

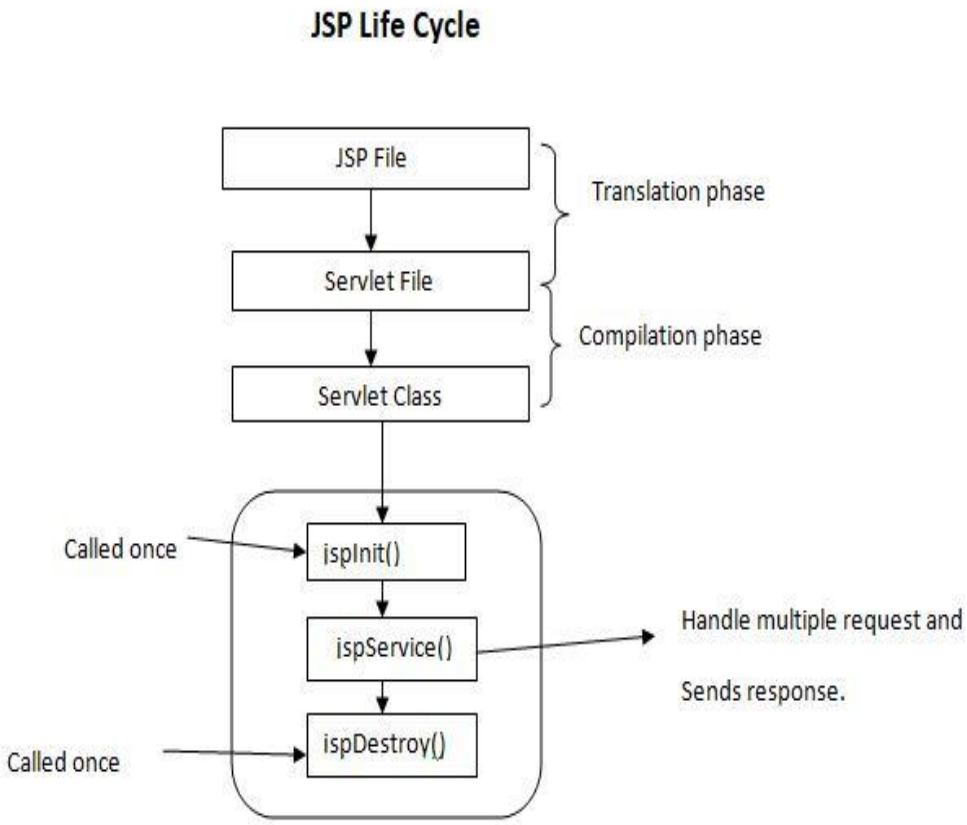
- Processes request
- Generates response
- Sends output to browser

6. Destruction Phase

- Method: destroy()
- Called when server is stopped or JSP is removed

Used for:

- Closing database connections
- Cleanup work



➢ JSP Installation

Required Software

- Eclipse IDE for Enterprise Java and Web Developers
- Java JDK (version 8 or above)
- Apache Tomcat 9.0

Step-by-Step Installation

Step 1: Install JDK

- Download and install JDK
- Set environment variables (JAVA_HOME)

Step 2: Install Eclipse

- Open Eclipse IDE
- Choose workspace

Step 3: Configure Tomcat Server

1. Go to **Window → Preferences**
2. Click **Server → Runtime Environment**
3. Click **Add**
4. Select Apache Tomcat (v9.0)
5. Browse Tomcat folder → Finish

Step 4: Create Dynamic Web Project

1. File → New → Dynamic Web Project
2. Enter Project Name: JSPDemo
3. Select Target Runtime (Tomcat)
4. Click Finish

Step 5: Create JSP File

1. Right click project → New → JSP File
2. Name it: index.jsp

Step 6: Run Project

- Right click → Run on Server
- Output will open in browser

➢ Comments in JSP

<%-- This is JSP comment --%>

Not visible in browser

➢ Complete JSP Example 1:

Step 1: Create Dynamic Web Project

1. Go to:File → New → Dynamic Web Project
2. Enter:Project Name: JSPDemo
3. Select Target Runtime (Tomcat)
4. Click **Next → Next → Finish**

Step 2: Create JSP File (index.jsp)

1. Right click on project →New → JSP File
2. File name:index.jsp
3. Click **Finish**

Step 3: Write Your JSP Code

Paste this code inside index.jsp:

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" %>

<html>
<body>

<h2>JSP Working Successfully</h2>

<p><%= "Hello JSP" %></p>

</body>
</html>
```

Step 4: Run Project

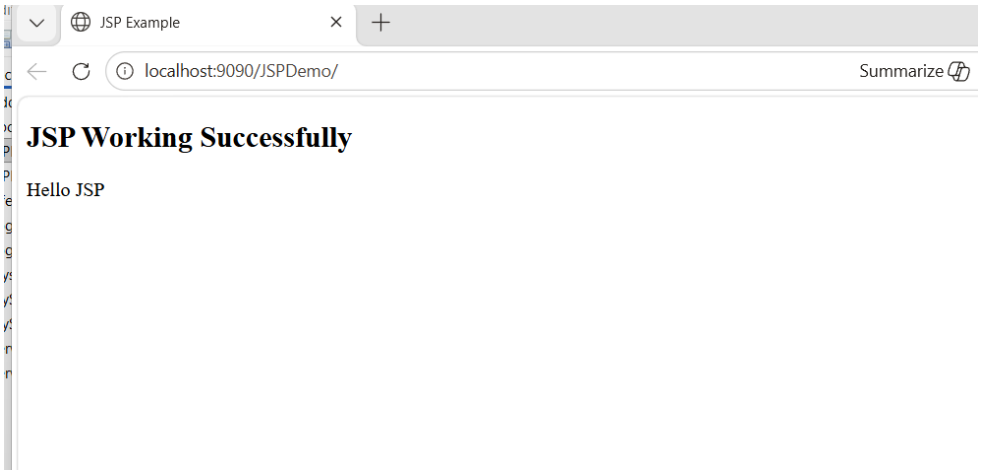
1. Right-click project
2. Select:Run As → Run on Server
3. Choose:Apache Tomcat
4. Click **Finish**

Step 5: Run JSP File

Open browser automatically OR manually go to:

http://localhost:8080/JSPDemo/index.jsp

Step 6: Output



Explanation:

1. JSP Page Directive

<%@ page language="java" contentType="text/html; charset=UTF-8" %>

Meaning:

- <%@ page %> → JSP directive (gives instructions to server)
- language="java" → JSP uses Java language
- contentType="text/html" → Output will be HTML page
- charset=UTF-8 → supports all characters (important for text)

2.Scriptlet Tag (Java Code)

<%= "Hello JSP" %>

<%= %> is an **expression tag**
It prints output directly to the browser

Here it prints: **Hello JSP**

4.2 JSP Tags & Components of JSP Page

➢ Components of JSP Page

A JSP page contains the following:

1. **Static Content (HTML)**
2. **JSP Directives**
3. **Scripting Elements**
4. **Action Tags**
5. **Implicit Objects**

Q.4 b) Explain JSP directives with example. [10 marks]

[May 2025]

1. JSP Directives

- Used to provide instructions to JSP container.
- **JSP Directives** are special instructions given to the JSP container (server like Apache Tomcat) to control how the JSP page is processed.
- They **do not produce output directly**, but they affect the **overall structure and behavior** of the JSP page.

✓ Syntax:

<%@ directive attribute="value" %>

Types of JSP Directives:

a) Page Directive

- The **page directive** is used to define **page-level settings** such as language, content type, import packages, error handling, etc.
- It defines page-level settings.
- Syntax:

```
<%@ page language="java" contentType="text/html" %>
```

Common Attributes of Page Directives

Attribute	Description
language	Specifies programming language (Java)
contentType	Defines MIME type
import	Imports Java packages
errorPage	Defines error page
isErrorPage	Specifies error handling page

b) Include Directive

- The **include directive** is used to include another file **at compile time**.
- It is useful for **header, footer, menu reuse**
- **Syntax:**

```
<%@ include file="header.jsp" %>
```

c) Taglib Directive

- The **taglib directive** is used to use **custom tags (libraries)** in JSP.
- Syntax:
  <%@ taglib uri="tag-library-uri" prefix="t" %>

Example 2: JSP Directives

STEP 1: Create Project

1. Open Eclipse
2. Go to:File → New → Dynamic Web Project
3. Project Name:JSPDirectiveDemo
4. Select **Apache Tomcat**
5. Click **Finish**

STEP 2: Create JSP Files

Create 3 files:

index.jsp
header.jsp
error.jsp

Right click project → **New** → **JSP File**

1. PAGE DIRECTIVE EXAMPLE

index.jsp

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" errorPage="error.jsp" %>
<%@ include file="header.jsp" %>

<!DOCTYPE html>
<html>
<head>
<title>Main JSP Page</title>
</head>
<body>

<h2>Main Page (index.jsp)</h2>

<p>This content is from index.jsp</p>

<hr>
```

```
<h3>Now generating an error intentionally:</h3>

<%
int a = 10;
int b = 0;
int c = a / b; // This will cause error
%>

</body>
</html>
```

error.jsp

```
<%@ page isErrorPage="true" %>

<!DOCTYPE html>
<html>
<head>
<title>Error Page</title>
</head>
<body>

<h2 style="color:red;">Error Occurred!</h2>

<p>Sorry, something went wrong.</p>

</body>
</html>
```

2. INCLUDE DIRECTIVE EXAMPLE

header.jsp

```
<!DOCTYPE html>

<html>
<body>

<h1 style="color:blue;">Welcome to JSP Directive Demo</h1>
<hr>

</body>
```

```
</html>
```

STEP 3: Run Project

1. Right click project
2. Click:Run As → Run on Server
3. Open browser:

http://localhost:8080/JSPAllDirectiveDemo/index.jsp

OUTPUTS

✓ Page Directive (Error Handling)

Error Occurred!
Sorry, something went wrong.

✓ Include Directive

Welcome Header
Main Page Content

2. Scripting Elements

Used to embed Java code in JSP.

a) Scriptlet Tag

- Contains **Java statements** (like loops, conditions, variable declarations).
- Code inside scriptlets is placed inside the _jspService() method during translation.
- **Syntax:**

```
<%
 Java code
 %>
```

Example:

```
<%
int a = 10;
int b = 20;
int sum=a+b
out.println("Sum = " + sum);
%>
```

b) Expression Tag

- Used to **output the result** of a Java expression directly to the browser.
- No semicolon (;) is required.
- Internally converted to out.print().
- **Syntax:**

```
<%= expression %>
```

- **Example:**

```
<%= "Welcome to JSP" %>
```

c) Declaration Tag

- Used to declare **variables and methods**.
- Code is placed **outside** the _jspService() method (at class level).
- **Syntax:**

```
<%!
 Declaration
 %>
```

- **Example:**

```
<%!
int count = 0;

int square(int n)
{
   return n * n;
}
%>
```

3. Action Tags

- JSP **Action Tags** are special XML-style tags used to perform actions like including files, forwarding requests etc.
- JSP Action Tags are used to perform dynamic tasks at runtime. Action tags run when the JSP page is requested, while directives run before execution (at translation time).

What are Action Tags?

- Written using <jsp:...> syntax
- Executed **at request time (runtime)**
- Used for **dynamic behavior** in JSP

Used to perform actions at runtime.

Common JSP Action Tags

1. <jsp:include>

Syntax:

```
<jsp:include page="file.jsp" />
```

Explanation:

- Includes another JSP/HTML file **during request time**
- Useful for dynamic content (e.g., header, footer)

Example:

```
<jsp:include page="header.jsp" />
<h2>Welcome to My Website</h2>
<jsp:include page="footer.jsp" />
```

Difference from include directive:

- <%@ include %> → compile time
- <jsp:include> → runtime

2. <jsp:forward>

Syntax:

```
<jsp:forward page="next.jsp" />
```

Explanation:

- Forwards request to another resource (JSP/Servlet)
- Stops execution of current page

Example:

```
<%
   <jsp:forward page="login.jsp" />

%>
```

3. <jsp:param>

Syntax:

```
<jsp:param name="key" value="value" />
```

Explanation:

- Used with <jsp:include> or <jsp:forward>
- Passes parameters to another resource

Example:

```
<jsp:forward page="welcome.jsp">
   <jsp:param name="username" value="Yogita" />
</jsp:forward>
```

➢ JSP Objects (Implicit Objects):

Explain any 5 implicit objects in JSP. [10 Marks]
[Previous year question paper Jan 2026]

Introduction

- Implicit objects in JSP are **built-in objects** that are automatically created by the JSP container (like Apache Tomcat).
- These objects help in handling **client requests, server responses, session tracking, and output generation** without explicitly creating them in code.
- They are available in all JSP pages by default.

1. request Object

Definition

- The request **object** is used to **receive data from the client** (browser).
- Whenever a user submits a form or sends data via URL, it is stored in the request object.
- It is an instance of:
  HttpServletRequest

What it does

- Gets form data
- Reads user input
- Retrieves request parameters
- Shares data between JSP pages

Common Methods

Method	Description
getParameter()	Gets form data
getParameterValues()	Gets multiple values
getAttribute()	Gets server-side data
setAttribute()	Stores data

Example (Getting Form Data)

```
<%
String name = request.getParameter("username");
out.println("Welcome " + name);
%>
```

If user enters:

username = Yogita

Output:

Welcome Yogita

Real-life Use

- Login forms
- Registration forms
- Search inputs

2. response Object

Definition

- The response **object** is used to **send data back to the client** (browser).
- It controls how the server responds to the browser.
- It is an instance of:
  HttpServletResponse

What it does

- Sends output to browser
- Redirects user to another page
- Sets content type

Common Methods

Method	Description
sendRedirect()	Redirect to another page
setContentType()	Sets response type
addCookie()	Adds cookies

Example (Redirect)

```
<%
response.sendRedirect("welcome.jsp");
%>
```

This will redirect user to **welcome.jsp**

Example (Content Type)

```
<%
response.setContentType("text/html");
out.println("Hello User");
%>
```

Real-life Use

- Redirect after login
- Sending file download
- Setting response format (HTML, JSON)

3. out Object

Type: JspWriter
Scope: Page

Explanation:

- The out object is used to **display data directly on the browser**.
- It works like System.out.println() but prints on web page instead of console.

Common Methods:

- println()
- print()
- flush()

Basic Syntax

```
out.print("Text");
out.println("Text");
```

Example:

```
<%
out.println("Hello JSP");
out.println("<h2>Welcome User</h2>");
%>
```

Output

Hello JSP
Welcome User

4. session Object

Definition

- The session object is used to **store user-specific data** across multiple pages.
- Each user has a separate session.
- The **session object** represents a **single user session**.

- It stores data specific to one user.
- It is an instance of:
  HttpSession

Key Features

- Unique for each user
- Created when user visits the site
- Destroyed after logout or timeout
- Used for **user-specific data**

Common Methods:

- setAttribute() → store data
- getAttribute() → retrieve data
- invalidate() → destroy session

Syntax

```
session.setAttribute("key", value);
session.getAttribute("key");
```

Example

```
<%
session.setAttribute("username", "Yogita");
%>

<p>
Welcome <%= session.getAttribute("username") %>
</p>
```

Output

Welcome Yogita

Real-life Use

- Login user information
- Shopping cart data
- User preferences

5. application Object

Definition

- The **application object** represents the entire web application.
- It is shared by **all users and all sessions**.
- It is an instance of:
  ServletContext

Key Features

- Available to **all users**
- Created when the application starts
- Destroyed when the application stops
- Used for **global data sharing**

Common Methods:

- setAttribute()
- getAttribute()
- removeAttribute()

Syntax

```
application.setAttribute("key", value);
application.getAttribute("key");
```

Example

```
<%
application.setAttribute("msg", "Welcome to JSP Application");
%>

<p>
<%= application.getAttribute("msg") %>
</p>
```

Output

Welcome to JSP Application

Real-life Use

- Website visitor counter
- Global configuration data
- Shared resources (DB connection info)

Difference Between Application and Session Object

Feature	Application Object	Session Object
Scope	Entire application	Single user
Shared by	All users	Only one user
Lifetime	App start → App stop	Login → Logout/Timeout
Data Type	Global data	User-specific data
Example	Visitor count	Username, cart

Advantages of Implicit Objects

✓ No need to create objects manually
✓ Reduces coding effort
✓ Makes JSP development faster
✓ Provides easy access to web components

Conclusion

Implicit objects are a **powerful feature of JSP** that simplify web development by providing ready-made objects for handling **request, response, session, and application-level operations**.

4.4 JDBC Connectivity in JSP

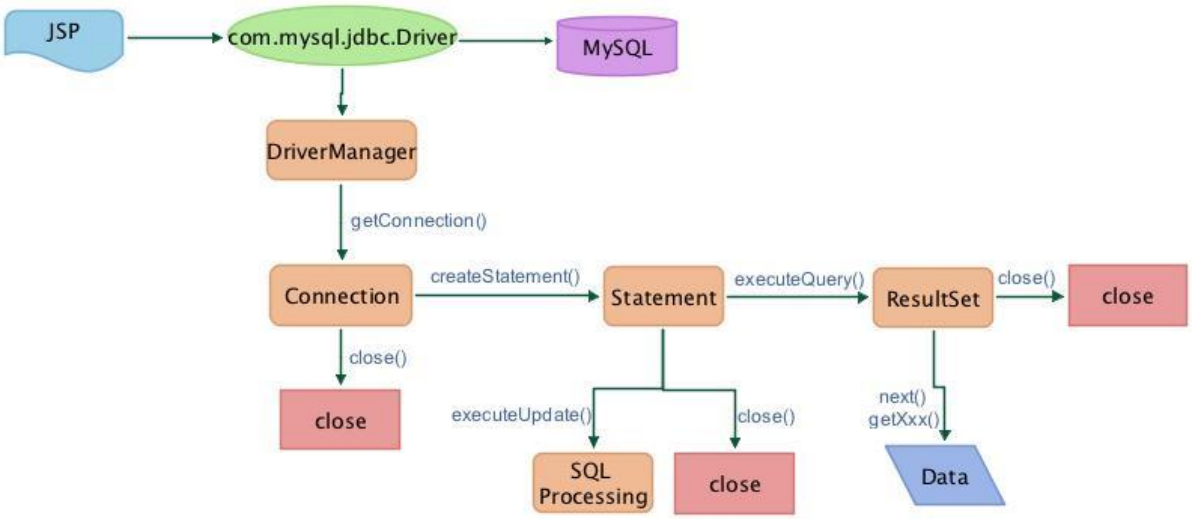
Introduction

- **JDBC (Java Database Connectivity)** is an API used to connect Java applications such as JSP with databases like MySQL, Oracle, etc.
- It enables execution of SQL queries and helps in retrieving and updating data from the database.
- **JSP (Java Server Pages)** is a server-side technology used to create dynamic web pages by embedding Java code within HTML.

Need of JDBC in JSP

- To connect web applications with database
- To store user data (forms, login details)
- To retrieve and display dynamic data
- To perform CRUD operations (Create, Read, Update, Delete)

JDBC Architecture



Working Flow:

1. Client sends request through browser
2. JSP page processes the request
3. JDBC API is used to connect to database
4. DriverManager loads the required driver
5. Database executes query
6. Result is returned to JSP and displayed to user

Steps for JDBC Connectivity

1. Import package (java.sql.*)
2. Load driver
3. Establish connection
4. Create statement
5. Execute query
6. Process result
7. Close connection

Important JDBC Components

- **DriverManager** → Loads database driver
- **Connection** → Establishes connection
- **Statement** → Executes SQL queries
- **PreparedStatement** → Secure and precompiled queries
- **ResultSet** → Stores query result

Advantages

✓ Platform independent
✓ Supports multiple databases
✓ Enables dynamic web applications
✓ Easy data handling

Disadvantages

❌Mixing Java code with HTML (in JSP)
❌Difficult to maintain
❌Security issues if not handled properly

Steps for JDBC Connectivity

⭑ **Step-by-Step Process:**

1. Import Packages

```
import java.sql.*;
```

2. Load Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

3. Establish Connection

```
Connection con = DriverManager.getConnection(
   "jdbc:mysql://localhost:3306/dbname", "username", "password");
```

4. Create Statement

```
Statement stmt = con.createStatement();
```

5. Execute Query

```
ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

6. Process Result

```
while(rs.next())
{
   System.out.println(rs.getInt(1) + " " + rs.getString(2));
}
```

7. Close Connection

```
con.close();
```

Example 1:

a) **Design Java Application (using JSP) for registration of hackthon (student name, Mail Id, Phone No, Gender, Course) and store data in appropriate table.**
[10 marks]

[Previous year paper question May 2025]

1. Aim

To develop a **JSP-based Hackathon Registration System** and store student details in MySQL database.

2. Requirements

- Java JDK
- Eclipse IDE
- Apache Tomcat
- MySQL Workbench
- MySQL Connector/J

3. Database Creation (Using test DB)

Step 1: Open MySQL Workbench

Step 2: Use Database

```
USE test;
```

Step 3: Create Table

```
CREATE TABLE hackathon
(
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100),
   email VARCHAR(100),
   phone VARCHAR(15),
   gender VARCHAR(10),
   course VARCHAR(50)
);
```

4. Project Creation in Eclipse

Steps:

1. Open Eclipse
2. File → New → Dynamic Web Project
3. Project Name: HackathonRegistration
4. Select Apache Tomcat
5. Finish

5. Add JDBC Driver

- Copy mysql-connector-j-9.6.0.jar
- Paste into:

WEB-INF/lib

Step-by-Step Procedure:

Method 1: Add JAR to Build Path

1. Right click your project
2. Click:
   Build Path → Configure Build Path
3. Go to:
   Libraries tab - select ClassPath
4. Click:
   Add External JARs
5. Select your file:

mysql-connector-j-9.6.0.jar

Click:
Apply → OK

✓ Now check:
Referenced Libraries
➞You should see your JAR file:mysql-connector-j-9.6.0.jar

Method 2:

Step 1:

- Right click on your project
- Example: HackathonRegistration

Step 2:

- Click on **Properties**

Step 3:

- In left side menu, select:
  Deployment Assembly

Step 4:

- Click on **Add** button

Step 5:

- Select option: Java Build Path Entries

Click **Next**

Step 6:

- You will see list of libraries
- Tick the checkbox:

mysql-connector-j-9.6.0.jar

Step 7:

- Click **Finish**

Step 8:

- Click **Apply → OK**

6.JSP Form Page

STEP : Create 2 JSP Files

register.jsp
save.jsp

Right click project → **New** → **JSP File**

register.jsp

```
<!DOCTYPE html>
<html>
<body>
<h2>Hackathon Registration</h2>

<form action="save.jsp" method="post">
Name: <input type="text" name="name"><br><br>
Email: <input type="email" name="email"><br><br>
Phone: <input type="text" name="phone"><br><br>

Gender:
<input type="radio" name="gender" value="Male">Male
<input type="radio" name="gender" value="Female">Female<br><br>

Course:
<select name="course">
<option>MCA</option>
<option>BCA</option>
<option>BBA</option>
</select><br><br>

<input type="submit" value="Register">
</form>

</body>
</html>
```

7. JSP Database Code (IMPORTANT CHANGE)

save.jsp

```
<%@ page import="java.sql.*" %>

<%
String name = request.getParameter("name");
String email = request.getParameter("email");
String phone = request.getParameter("phone");
String gender = request.getParameter("gender");
String course = request.getParameter("course");

try {
Class.forName("com.mysql.cj.jdbc.Driver");

Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/test?useSSL=false&serverTimezone=UTC",
"root",
"root"
);

PreparedStatement ps = con.prepareStatement(
"INSERT INTO hackathon(name,email,phone,gender,course)
VALUES(?,?,?,?,?)"
);

ps.setString(1, name);
ps.setString(2, email);
ps.setString(3, phone);
ps.setString(4, gender);
ps.setString(5, course);

ps.executeUpdate();

out.println("<h2>Registration Successful!</h2>");

con.close();

} catch(Exception e) {
out.println(e);
}
%>
```

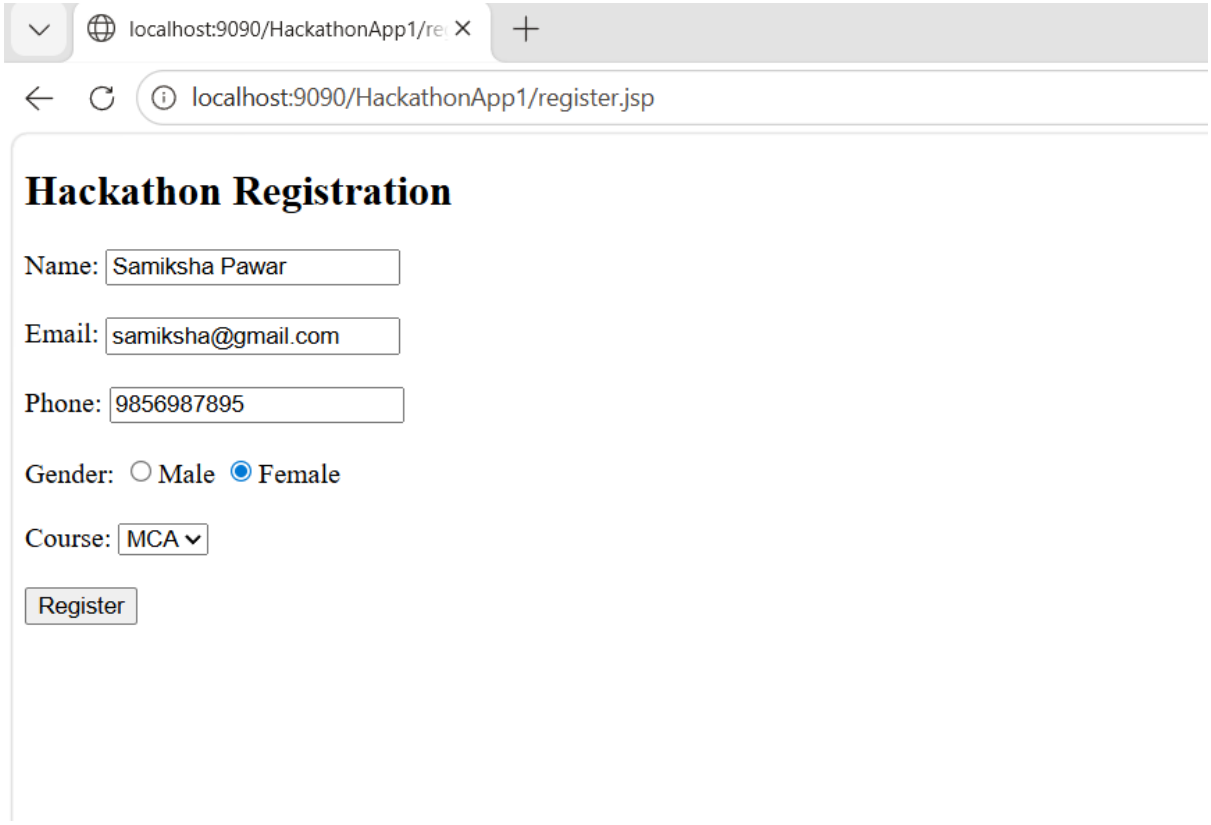
8. Execution Steps

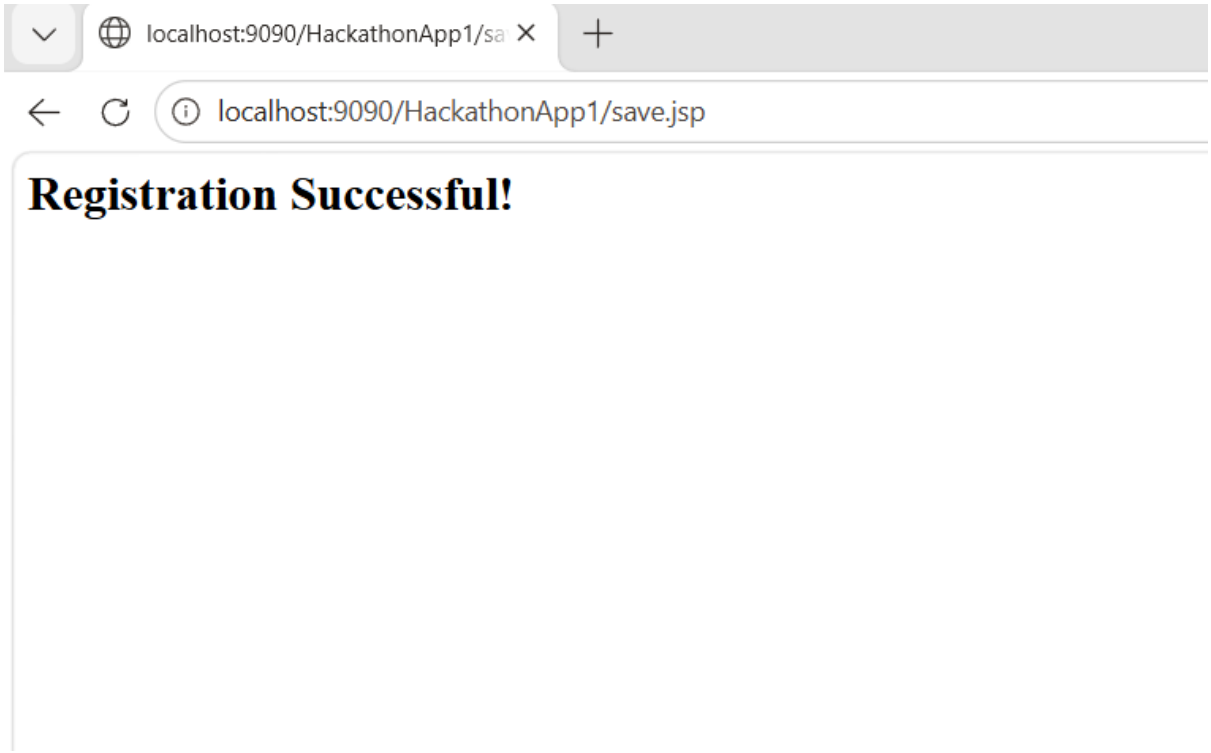
1. Right click project
2. Run on Server
3. Select Apache Tomcat

Open:

http://localhost:8080/HackathonRegistration/register.jsp

9. Output



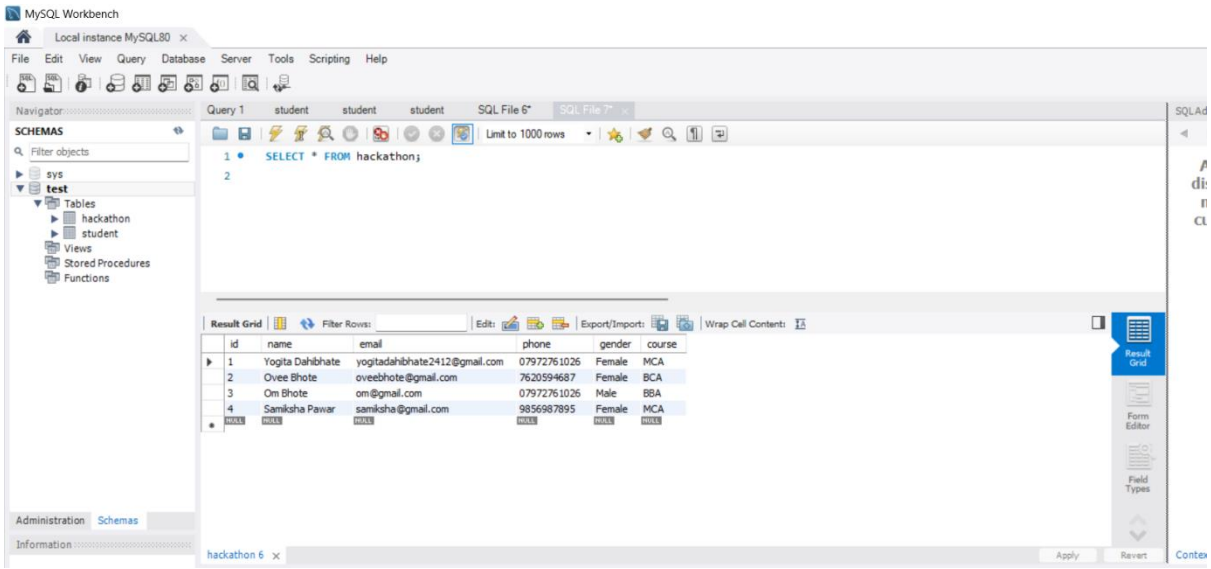


9. Verify Data in Database

Open MySQL Workbench

Run:

SELECT * FROM hackathon;



Example 2:

Create a jsp page to accept rollno of student and update mobile number in student a table. display updated details. assume suitable structure.

[10 marks] [Previous year Question paper Jan 2026]

1. Aim

To create a JSP application that accepts **roll number**, updates **mobile number** in database, and displays updated student details.

2. Requirements

- Java JDK
- Eclipse IDE
- Apache Tomcat
- MySQL Workbench
- MySQL Connector/J

3. Database Design

Step 1: Use Database

```
USE test;
```

Step 2: Create Table (Student)

```
CREATE TABLE student (
   rollno INT PRIMARY KEY,
   name VARCHAR(100),
   mobile VARCHAR(15),
   course VARCHAR(50)
);
```

Step 3: Insert Sample Data

```
INSERT INTO student VALUES
(1,'Sachin','9876543210','MCA'),
(2,'Ravi','9123456780','BCA');
```

4. Project Creation in Eclipse

Steps:

6. Open Eclipse
7. File → New → Dynamic Web Project
8. Project Name: StudentApp
9. Select Apache Tomcat
10. Finish

5. Add JDBC Driver

- Copy mysql-connector-j-9.6.0.jar
- Paste into:

WEB-INF/lib

Step-by-Step Procedure:

Method 1: Add JAR to Build Path

6. Right click your project
7. Click:
   Build Path → Configure Build Path
8. Go to:
   Libraries tab - select ClassPath
9. Click:
   Add External JARs
10. Select your file:

mysql-connector-j-9.6.0.jar

Click:
Apply → OK

✓ Now check:
Referenced Libraries
➞You should see your JAR file:mysql-connector-j-9.6.0.jar

Method 2:

Step 1:

- Right click on your project
- Example: StudentApp

Step 2:

- Click on **Properties**

Step 3:

- In left side menu, select:
  Deployment Assembly

Step 4:

- Click on **Add** button

Step 5:

- Select option: Java Build Path Entries

Click **Next**

Step 6:

- You will see list of libraries
- Tick the checkbox:

mysql-connector-j-9.6.0.jar

Step 7:

- Click **Finish**

Step 8:

- Click **Apply → OK**

5. JSP Page (Input Form)

update.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
<body>

<h2>Update Mobile Number</h2>

<form action="updateProcess.jsp" method="post">
Roll No: <input type="text" name="rollno"><br><br>
New Mobile: <input type="text" name="mobile"><br><br>

<input type="submit" value="Update">

</form>

</body>
</html>
```

6. JSP Processing Page (Update + Display)

updateProcess.jsp

```
<%@ page import="java.sql.*" %>

<%
int rollno = Integer.parseInt(request.getParameter("rollno"));
String mobile = request.getParameter("mobile");

try
{
Class.forName("com.mysql.cj.jdbc.Driver");

Connection con =
DriverManager.getConnection( "jdbc:mysql://localhost:3306/test", "root",
"root" );

// □ Update Query
PreparedStatement ps = con.prepareStatement("UPDATE student SET mobile=?
WHERE rollno=?" );
```

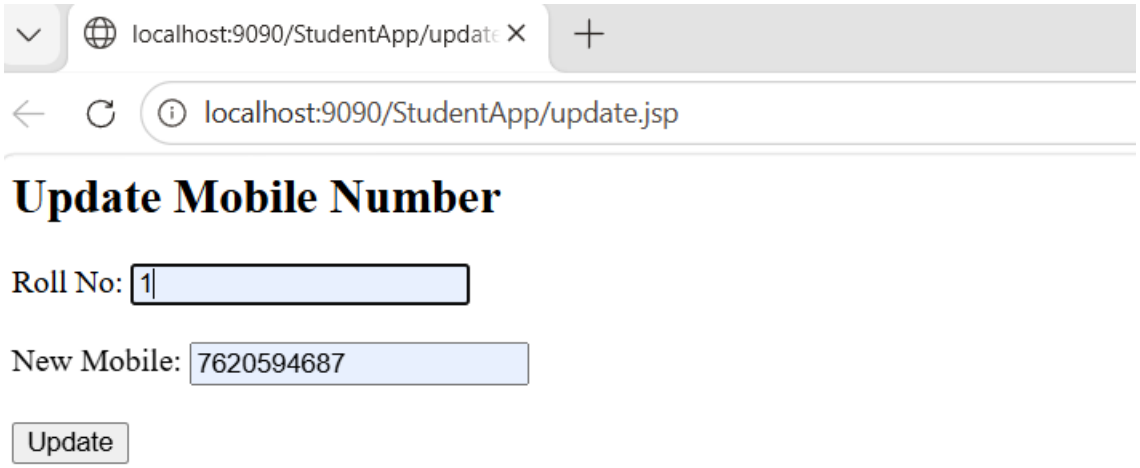
```
ps.setString(1, mobile);
ps.setInt(2, rollno);

int i = ps.executeUpdate();

if(i > 0)
{
out.println("<h3>Mobile Updated Successfully</h3>");

// □ Fetch Updated Record
PreparedStatement ps2 = con.prepareStatement( "SELECT * FROM student
WHERE rollno=?");

ps2.setInt(1, rollno);
ResultSet rs = ps2.executeQuery();

while(rs.next())
{
%>

<h3>Updated Details:</h3>
Roll No: <%= rs.getInt("rollno") %><br>
Name: <%= rs.getString("name") %><br>
Mobile: <%= rs.getString("mobile") %><br>
Course: <%= rs.getString("course") %><br>

<%
}
}
else
{
out.println("<h3>Record Not Found</h3>");
}

con.close();

} catch(Exception e){
out.println(e);
}
%>
```

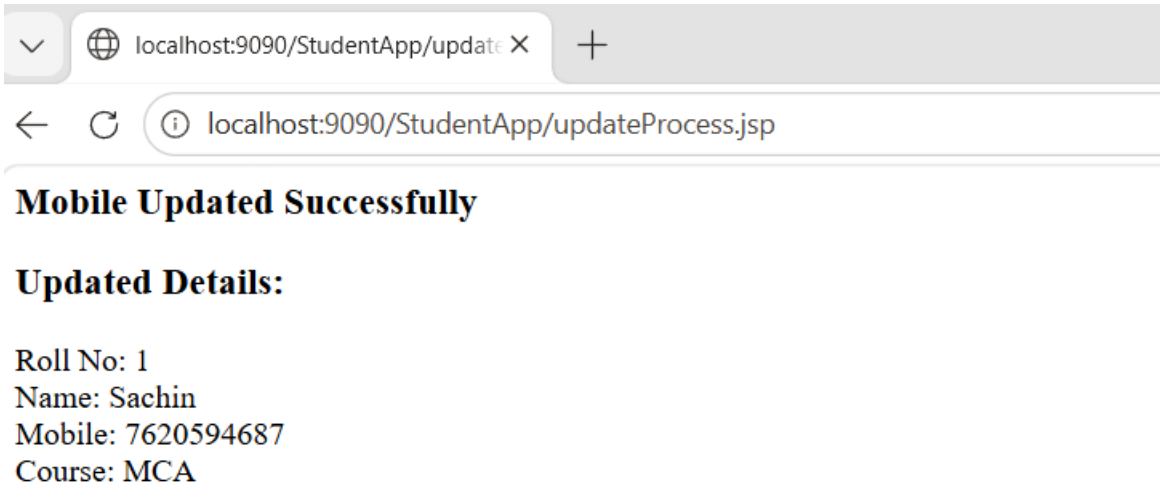
7. Execution Steps

1. Right click project → Run on Server
2. Select Apache Tomcat
3. Open:

http://localhost:8080/YourProjectName/update.jsp

8. Output





9. Verification in Database

Open MySQL Workbench

Run:

SELECT * FROM student;